

World Partition 핵심 개념 — 발표문

슬라이드별 발표 스크립트 · 예상 소요 12~15분 · 스터디/발표용

01 · 타이틀

안녕하세요. 오늘은 언리얼 엔진 5의 오픈 월드 제작 기반인 **World Partition**의 핵심 개념을 정리해 보겠습니다. 이 자료는 Epic Games Japan의 스즈키 타카시가 발표한 「World Partition 2024」를 바탕으로 재구성했습니다. 세부 기능 나열보다는 "왜 이 시스템이 필요했고, 무엇으로 구성되어 있는가"라는 기본 개념에 집중해서 보겠습니다.

02 · 등장 배경

먼저 World Partition이 왜 등장했는지부터 보겠습니다. UE3~UE4 시절에는 **단일 Persistent Level** 하나에 모든 걸 배치했습니다. 문제는 레벨 파일이 통째로 하나라서 동시에 한 명만 편집할 수 있었다는 점입니다. UE4에서는 **Sublevels Workflow**로 이걸 개선했습니다. 영역별로 Sublevel을 나눠 각자 잠그고 작업하고, 동적 로드로 심리스한 월드도 구현했죠. 하지만 어디까지나 **수동 분할**이었습니다. 2010년대 이후 오픈 월드의 규모가 커지면서 분할해야 할 레벨 수가 폭증했고, 사람이 손으로 관리하는 방식은 한계에 부딪혔습니다. World Composition이라는 도구도 있었지만 역부족이었고요. 그래서 2022년 4월, UE5와 함께 World Partition이 등장합니다.

03 · World Partition 개요

World Partition의 특징은 세 가지로 요약됩니다. 첫째, **월드 자동 분할** — 지정한 크기의 그리드로 World를 자동으로 나눠 주기 때문에 수동 레벨 분할 작업이 사라집니다. 둘째, **레벨 동시 편집** — One File Per Actor 덕분에 여러 개발자가 같은 레벨을 동시에 편집할 수 있습니다. 셋째, **자동 스트리밍** — 거리 기반으로 Cell을 자동 로드/언로드해서 심리스한 오픈 월드를 만들어 줍니다. 핵심 아이디어를 한 문장으로 정리하면 이렇습니다: **에디터에서는 하나의 거대한 World로 편집하고, PIE나 Cook 시점에 스트리밍 가능한 Cell로 자동 분할된다.**

04 · 구성 요소

중요한 점은 World Partition이 하나의 큰 기능이 아니라 **여러 도구와 기능의 조합**이라는 겁니다. 오늘 다룰 여섯 가지는 다음과 같습니다. 분할과 제어의 기본 메커니즘인 **World Partition Grid**, 동시 편집을 가능하게 하는 **One File Per Actor**, Actor를 레이어 단위로 관리하는 **Data Layer**, 원거리를 간략화 메시로 대체하는 **HLOD**, 레벨을 하나의 Actor로 다루는 **Level Instance**, 그리고 프로시저럴 배치 도구인 **PCG**입니다. 이제 하나씩 살펴보겠습니다.

05 · Cell 분할과 스트리밍

먼저 그리드 분할입니다. World는 기본적으로 **2D 그리드**로 자동 분할되고, 이 분할 단위를 **Cell**이라고 부릅니다. 화면의 그림처럼 월드 위에 격자가 그려지고, 플레이어를 중심으로 한 로딩 범위 안의 Cell만 메모리에 올라갑니다. 주의할 점은, 분할선이 메시지를 물리적으로 자르는 게 아니라 **Actor 단위로 어느 Cell에 속하는지 할당**된다는 겁니다. Cell의 크기, 로딩 범위, 우선순위는 World Settings에서 설정합니다. 그리고 실무적으로 중요한 주의사항 하나 — 기존 레벨을 World Partition으로 변환하는 건 **비가역**입니다. 한 번 변환하면 되돌릴 수 없으니 신중하게 결정해야 합니다.

06 · Streaming Source

스트리밍의 기준점이 되는 것이 **Streaming Source**입니다. 기본적으로 플레이어 중심으로 로드되는 이유는, **Player Controller 자체가 Streaming Source로 취급**되기 때문입니다. 화면의 디버그 표시를 보시면 캐릭터가 이동함에 따라 주변 Cell이 자동으로 로드되는 걸 확인할 수 있습니다.

Streaming Source는 **Component**라서 임의의 Actor에 부착할 수 있고, 로드 대상 필터나 범위 Shape을 파라미터로 지정합니다. 대표적인 활용처가 **Fast Travel**입니다. 순간이동할 목적지 주변 Cell을 미리 로드해 두면 로딩 화면 없이 전환할 수 있죠. 다만 로드하는 Cell이 늘어나는 만큼 메모리도 늘어나니, 목적지 근처 최소 범위만 로드하도록 제어하는 게 좋습니다.

07 · 쿼드트리와 옥트리

여기서 잠깐, World Partition의 바탕에 있는 **공간 분할** 개념을 짚고 가겠습니다. 월드에는 수천에서 수십만 개의 오브젝트가 있는데, 매 프레임 이것 전부 검사할 수는 없습니다. 그래서 공간을 쪼개 필요한 영역만 처리하는 것이 공간 분할입니다. **쿼드트리(QuadTree)**는 2D 공간을 4개 영역으로 재귀적으로 나누는 구조입니다. 오브젝트가 밀집된 곳만 더 세분화하기 때문에, 전체를 선형 탐색하면 $O(N)$ 인 것이 $O(\log N)$ 수준으로 줄어듭니다. 충돌 검사, 시야 판정, RTS 유닛 탐색 등에 널리 쓰입니

다. **옥트리(Octree)**는 이것의 3차원 버전으로, 3D 공간을 8개 영역으로 나눕니다. 언리얼 엔진 내부에서도 성능 최적화에 옥트리 기반 구조를 활용하고 있습니다. World Partition의 그리드 분할과 스트리밍도 결국 이 공간 분할 개념 위에서 "필요한 영역만 로드한다"를 구현한 것입니다.

08 · One File Per Actor

다음은 동시 편집을 가능하게 하는 **One File Per Actor**, 줄여서 OFPA입니다. 원래는 레벨에 배치된 모든 Actor 정보가 .umap 파일 하나에 통째로 저장됐습니다. OFPA는 이것 **배치된 Actor 인스턴스 하나당 파일 하나**로 분리합니다. 그래서 소스 컨트롤에서 Actor 단위로 잠그고 편집할 수 있고, 여러 명이 같은 레벨을 동시에 작업할 수 있게 됩니다. External Actors라고도 불립니다. 파일은 해시명으로 __ExternalActors__ 폴더에 분산 저장되는데, Content Browser에는 보이지 않습니다. 그래서 문제가 생긴 파일의 해시명에서 실제 Actor를 찾아야 할 때가 있는데, 방법은 세 가지입니다. 아웃라이너 필터에 해시명을 붙여넣어 Package Short Name 컬럼과 매칭하거나, 소스 컨트롤의 View Change 대화상자에서 경로를 확인하거나, PIE·Cook 시 출력되는 분할 로그에서 확인하면 됩니다.

09 · Data Layer

Data Layer는 레벨 스트리밍에 대한 필터입니다. 페인트 소프트웨어의 레이어처럼 Actor를 레이어에 할당해서 관리한다고 생각하시면 됩니다. 화면의 예시처럼 타일 하나를 Landscape, Static Geometry, Resources, POIs, Caves 같은 레이어로 분해할 수 있습니다. 종류는 두 가지입니다. **Runtime Data Layer**는 런타임에 임의 타이밍의 로드/언로드를 제어합니다. 퀘스트 진행이나 이벤트 조건에 따라 Actor 그룹을 등장시키거나 사라지게 할 수 있죠. **Editor Data Layer**는 에디터 편집 지원 전용입니다. 지금 작업과 무관한 Actor를 숨겨서 쾌적하게 작업하기 위한 것으로, 런타임에는 존재하지 않습니다.

10 · Level Instance

Level Instance는 레벨을 통째로 하나의 Actor처럼 다루는 기능입니다. 왜 쓰는지 세 가지로 정리하면 — 첫째, **편집 효율과 재사용성**입니다. 건물이나 특정 영역을 하나의 Actor로 묶어 다른 World에 배치할 수 있고, 같은 레벨을 여러 개 복제하는 Prefab식 재사용이 가능합니다. 둘째, **일괄 변경 전파**입니다. Ctrl+E로 인라인 편집에 들어가 수정하고 커밋하면 모든 인스턴스에 한 번에 적용됩니다. 배치된 복사본을 일일이 고칠 필요가 없죠. 셋째, **부가 활용**으로 작업 상태의 스냅샷 저장 용도로도 쓸 수 있고, Actor가 많은 모듈 구조라면 Packed Level Actor로 ISM/HISM 최적화 효과도 얻을 수 있습니다. 화면처럼 편집 모드에서는 편집 중인 인스턴스만 컬러로, 나머지는 회색으로 표시됩니다.

11 · HLOD

HLOD, Hierarchical Level of Detail은 원거리를 간략화된 메시로 대체하는 계층 LOD입니다. 쓰는 이유는 명확합니다 — **Draw Call을 줄여 프레임을 최적화**하기 위해서입니다. HLOD는 World Partition Cell마다 작성되어, Static Mesh를 가진 HLOD Actor로 레벨 안에 배치됩니다. 핵심 개념은 세 가지입니다. 첫째, 셀의 디스플레이 관련 구성 요소만 모아 **단일 액터로 결합**합니다. 둘째, 셀이 로딩 범위를 벗어나 스트림 아웃되면 **그 자리에 HLOD Actor가 대신 표시**됩니다. 그래서 멀리 있는 지형과 건물이 계속 보이는 겁니다. 셋째, 런타임에 생성되는 게 아니라 **사전에 정적으로 빌드**해야 합니다. 에디터에서 Build HLOD를 실행하거나 Commandlet으로 변환 처리합니다. 화면의 두 이미지를 비교해 보시면, 왼쪽이 완전한 디테일의 원본 에셋이고 오른쪽이 그리드 셀에 의해 생성된 HLOD 에셋입니다. 멀리서 보면 차이를 느끼기 어렵지만 비용은 크게 줄어듭니다.

12 · PCG

마지막 구성 요소는 **PCG**, Procedural Content Generation입니다. Blueprint 같은 비주얼 스크립팅으로 규칙을 작성하면, 그 규칙에 따라 Actor를 자동으로 배치해 주는 프로시저럴 기능입니다. 광대한 World에 나무, 바위, 풀 같은 에셋을 자연스럽게 수작업으로 배치하는 건 사실상 불가능에 가깝는데, PCG를 쓰면 작업 효율이 극적으로 올라갑니다. 화면의 예시는 스폴라인으로 지정한 영역 안에 나무·바위·길이 자동 생성된 모습입니다. 실제로 **레고 포트나이트**가 PCG를 대대적으로 활용해 제작됐고, 발표자는 "이것이 없었다면 완성하지 못했을 것"이라고 말할 정도입니다.

13 · 참고 자료 (마무리)

정리하겠습니다. World Partition은 **자동 분할, 동시 편집, 자동 스트리밍**이라는 세 가지 축으로 오픈 월드 제작의 패러다임을 바꾼 시스템이고, Grid·OFPA·Data Layer·Level Instance·HLOD·PCG라는 도구들의 조합으로 이루어져 있습니다. 더 깊게 보실 분들을 위해 참고 자료 세 가지를 남깁니다. 원 발표의 한국어 번역본, Epic 공식 World Building Guide, 그리고 UEFN의 Streaming and HLODs 문서입니다. 이상으로 발표를 마치겠습니다. 질문 있으시면 편하게 해 주세요.